UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

Ingeniería de Requisitos

Sílabo y material de estudio

Código de la asignatura: **ISR-401**

Cuarto nivel — 3 créditos — 96 horas presenciales

*Material de estudio elaborado conforme al estándar internacional **IEEE/ISO/IEC 29148:2018**, la guía **SWEBOK v4.0 (2024)**, los modelos de calidad **ISO/IEC 25010:2023** y la segunda edición del manual de Pohl (Springer, 2024).*

Período académico: 2025 – 2026

Quevedo – Los Ríos – Ecuador

«La especificación de requisitos es el momento de la ingeniería del software en el que se decide, en última instancia, si el sistema va a resolver el problema correcto. Todo lo que viene después es construcción; lo que viene antes es ingeniería de requisitos. »

— *adaptado de Klaus Pohl [**Pohl2024**]*

Aviso: Este material acompaña al *Programa Analítico* oficial de la asignatura aprobado por la Facultad de Ciencias de la Computación. Los contenidos están enmarcados en los estándares y guías internacionales que se citan a lo largo del texto y se sustentan en bibliografía verificable, según las normas IEEE.

Índice general

Índice de figuras

Índice de tablas

Semana 1

Ingeniería de Requisitos en procesos ágiles

Los métodos ágiles no eliminan los requisitos; eliminan el documento de requisitos monolítico. Los requisitos siguen existiendo, solo que viven en el backlog, en las historias de usuario y — sobre todo — en las conversaciones continuas entre el equipo y el cliente.
adaptado de **Cohn2004**

Objetivos de aprendizaje de la semana

Al finalizar esta decimocuarta semana, el estudiante estará en capacidad de:

1. Gestionar el **product backlog** como artefacto central de IR ágil, con épicas, historias de usuario y criterios de aceptación [**Schwaber2020**, **Cohn2004**, **IREB_CPRE_2024**].
2. Aplicar **BDD** (Behavior Driven Development) para convertir los criterios de aceptación en especificaciones ejecutables con Gherkin/Cucumber [**Adzic2011**, **North2006**, **Beck2001**].
3. Integrar las actividades de IR con los **eventos y artefactos de Scrum**: Sprint Planning, Refinement, Review y retrospectiva [**Schwaber2020**, **Pohl2024**, **IREB_CPRE_2024**].
4. Comparar **IR tradicional vs. IR ágil** y decidir cuándo aplicar cada enfoque en el PFC [**Pohl2024**, **Cohn2004**, **Schwaber2020**].

1.1 Subtema 14.1. IR en procesos ágiles: backlog, historias y criterios

1.1.1 El product backlog como ERS ágil

Definición

Product Backlog. Lista ordenada y emergente de todo lo que se conoce que es necesario en el producto. Es el único artefacto de requisitos en Scrum y el responsable de su calidad es el **Product Owner** (PO). No es una lista estática: se refina continuamente en cada sprint [**Schwaber2020**, **Cohn2004**].

La jerarquía del backlog en un PFC ágil se compone de tres niveles:

Tabla 1.1. *Jerarquía de artefactos del backlog ágil y su equivalente en IR tradicional* [**Cohn2004**, **Schwaber2020**, **Pohl2024**].

Nivel ágil	Descripción	Equivalente IR tradicional	Ejemplo (PFC camaroneras)
Épica	Capacidad grande que tarda más de un sprint.	Módulo o subsistema del ERS.	*Monitoreo de calidad del agua en tiempo real.*
Historia	Entregable en un sprint; criterio INVEST.	RF individual en EARS.	*Como técnico, quiero ver la temperatura de la piscina 3 en el dashboard.*
Tarea	Unidad de trabajo técnico dentro de la historia.	Subtarea de diseño o implementación.	*Crear endpoint REST GET /piscinas/id/sensores.*

Refinamiento del backlog: de épica a historia INVEST

El **Backlog Refinement** (o Grooming) es el evento de Scrum donde el PO y el equipo descomponen épicas en historias, añaden criterios de aceptación y estiman el esfuerzo en puntos de historia [**Schwaber2020**]. Para el PFC de camaroneras:

Ejemplo en Ecuador

Refinamiento de épica → historias INVEST (PFC camaroneras).
Épica: Monitorear calidad del agua en tiempo real.
Historia US-C01 (Must-have, 5 puntos): *Como técnico acuícola, quiero ver en el dashboard el valor de oxígeno disuelto de cada piscina actualizado cada 30 segundos, para detectar condiciones críticas antes de que los camarones comiencen a morir.*
Criterios de aceptación (Gherkin):

- **Dado** que el dashboard está abierto y el sensor de la piscina 3 está activo,
  cuando la lectura de O2 cambia en el sensor,
  entonces el dashboard muestra el nuevo valor en menos de 35 segundos.
- **Dado** que el O2 cae por debajo de 4.0 mg/L,
  cuando el sistema detecta la lectura crítica,
  entonces el dashboard muestra la alerta en rojo y suena la alarma sonora.

Historia US-C02 (Must-have, 3 puntos): *Como administrador, quiero exportar el historial de lecturas de oxígeno de los últimos 30 días por piscina en formato CSV, para el reporte semanal a la extractora.*

1.2 Subtema 14.2. BDD: especificación ejecutable de requisitos

Definición

Behavior Driven Development (BDD). Técnica de desarrollo propuesta por Dan North [**North2006**] que extiende el TDD (Test-Driven Development) al nivel de los requisitos: los criterios de aceptación se escriben como **escenarios Gherkin** *antes* de implementar la historia, y se convierten automáticamente en pruebas ejecutables con frameworks como Cucumber (Java/Ruby), Behave (Python) o Pytest-BDD (Python) [**Adzic2011**, **North2006**, **Beck2001**].

1.2.1 La anatomía completa de un escenario Gherkin

Tabla 1.2. *Palabras clave de Gherkin y su semántica [***North2006***,* **Adzic2011***].*

Palabra clave	Semántica	En el PFC de mascotas
Feature	El nombre de la funcionalidad (épica).	`Feature: Reservar servicio de paseo.`
Background	Precondiciones comunes a todos los escenarios del Feature.	`Given el dueño ha iniciado sesión.`
Scenario	Un caso de prueba de aceptación concreto.	`Scenario: Reserva exitosa con cuidador disponible.`
Given	Estado del sistema antes de la acción.	`Given existe un cuidador certificado disponible.`
When	La acción del usuario o del sistema.	`When el dueño selecciona fecha, hora y cuidador.`
Then	El resultado observable verificable.	`Then el sistema confirma la reserva y envía SMS.`
And / But	Condiciones adicionales encadenadas.	`And el cuidador recibe notificación push.`
Scenario Outline	Escenario parametrizado con tabla de ejemplos.	`Outline: Reservar servicio de tipo <tipo>.`

Feature file completo: PFC mascotas a domicilio

Ejemplo en Ecuador

Archivo `reserva_paseo.feature` — PFC Mascotas.

```
Feature: Reservar servicio de paseo de mascota

  Background:
    Given el dueño "Carlos Ponce" ha iniciado sesión
    And tiene registrada la mascota "Simba" (golden retriever, 4 años)

  Scenario: Reserva exitosa con cuidador disponible
    Given existe el cuidador "María Loor" disponible el 2025-06-15 a las 08h00
    When Carlos selecciona a María para el paseo de 60 minutos
    And confirma la reserva
    Then el sistema registra la reserva con estado CONFIRMADO
    And envía SMS a Carlos: "Reserva confirmada con María el 15/06 08h00"
    And envía notificación push a María con los datos de Simba

  Scenario: Sin cuidadores disponibles
    Given no hay cuidadores disponibles el 2025-06-15 a las 08h00
    When Carlos intenta reservar a esa hora
    Then el sistema muestra "No hay cuidadores disponibles en ese horario"
    And sugiere los 3 horarios más próximos con disponibilidad

  Scenario Outline: Reservar distintos tipos de servicio
    When Carlos selecciona el servicio "<tipo>"
    Then el sistema aplica la tarifa de <precio> dólares por hora
    Examples:
      | tipo          | precio |
      | Paseo         | 5.00   |
      | Guardería día | 8.00   |
      | Vet. domicilio| 15.00  |
```

1.2.2 De Gherkin a prueba ejecutable con Pytest-BDD

Ejemplo en Ecuador

Step definitions en Python (Pytest-BDD) para el PFC mascotas.

```
# test_reserva_paseo.py
from pytest_bdd import given, when, then, scenario

@scenario('reserva_paseo.feature',
          'Reserva exitosa con cuidador disponible')
def test_reserva_exitosa():
    pass

@given('existe el cuidador "María Loor" disponible el 2025-06-15 a las 08h00')
def cuidador_disponible(db):
    db.insert_cuidador('María Loor', fecha='2025-06-15', hora='08:00')

@when('Carlos selecciona a María para el paseo de 60 minutos')
def seleccionar_cuidadora(client, reserva_data):
    reserva_data['response'] = client.post('/api/reservas', json={
        'cuidador_id': 'maria-loor',
        'duracion_min': 60,
        'fecha': '2025-06-15',
        'hora': '08:00'
    })

@then('el sistema registra la reserva con estado CONFIRMADO')
def verificar_estado(reserva_data):
    assert reserva_data['response'].json()['estado'] == 'CONFIRMADO'
```

1.3 Subtema 14.3. Integración de IR con Scrum

1.3.1 El rol del Product Owner en la IR ágil

En Scrum, el **Product Owner** es el responsable único del Product Backlog y actúa como el analista de requisitos del equipo [**Schwaber2020**]. Sus responsabilidades de IR son:

- Redactar o aprobar todas las historias de usuario antes de que entren al Sprint Planning.
- Mantener el backlog priorizado por valor de negocio (no por preferencias técnicas del equipo).
- Participar en el Refinement al menos 2 horas por semana.
- Definir el criterio de **Definition of Done** (DoD) que incluye al menos: criterios GWT pasados, RTM actualizada y documentación del usuario actualizada.
- Aceptar o rechazar el incremento en la Sprint Review.

1.3.2 IR en cada evento Scrum

Tabla 1.3. *Actividades de IR en cada evento de Scrum* [**Schwaber2020**, **Pohl2024**, **IREB_CPRE_2024**].

Evento Scrum	Actividad de IR	Artefacto producido
Sprint Planning	El PO presenta las historias del sprint; el equipo verifica INVEST y estima esfuerzo.	Sprint Backlog con historias listas para desarrollar.
Daily Scrum	¿Hay bloqueos por ambigüedad de requisitos? El PO resuelve en el día.	Decisión de aclaración documentada en el issue de Jira.
Backlog Refinement	Desglose de épicas, escritura de criterios GWT, estimación.	Historias refinadas con criterios de aceptación.
Sprint Review	El PO valida el incremento contra los criterios de aceptación.	Historias aceptadas/rechazadas; nuevas historias si surgen.
Retrospectiva	¿El proceso de escritura de historias funcionó? ¿Hubo muchos rechazos por criterios mal escritos?	Mejoras al proceso de IR del equipo.

1.4 Subtema 14.4. XP y comparación IR tradicional vs. IR ágil

1.4.1 IR en Extreme Programming (XP)

Extreme Programming (XP) [**Beck2001**] tiene su propio conjunto de prácticas de IR que difieren de Scrum pero comparten el énfasis en la comunicación continua:

- **Customer on site**: el cliente (o su representante) está físicamente presente con el equipo para responder preguntas de requisitos en tiempo real. En el PFC, el dueño del negocio puede participar en sesiones semanales presenciales de 2 horas.
- **User stories**: el cliente escribe las historias en tarjetas físicas o en Jira; el equipo las estima; se negocian alcance y prioridad en cada iteración.
- **Acceptance tests**: el cliente escribe los criterios de aceptación antes de la implementación (similar a BDD).
- **Small releases**: entregas cada 1–2 semanas validan continuamente que el sistema en construcción coincide con lo que el cliente necesita.

1.4.2 Comparación IR tradicional vs. IR ágil

Tabla 1.4. *Comparación sistemática entre IR tradicional e IR ágil [***Pohl2024***,* **Cohn2004***,* **Schwaber2020***,* **IREB_CPRE_2024***].*

Dimensión	IR Tradicional	IR Ágil
Artefacto central	ERS/SRS con los 10 atributos IEEE 29148.	Product Backlog con historias INVEST.
Momento de especificar	Antes del desarrollo; baseline aprobado.	Iterativo; just-in-time por sprint.
Criterio de calidad	Densidad de defectos; cobertura RTM.	Criterios GWT; Definition of Done.
Gestión del cambio	Proceso CCB formal; CR documentado.	Reordenamiento del backlog; PO decide.
Participación del cliente	Revisiones planificadas en hitos.	Continua; Review cada sprint.
Trazabilidad	RTM bidireccional formal.	Links en Jira; BDD tests.
Cuándo aplicar	Alta criticidad; contrato fijo; regulado.	Requisitos cambiantes; cliente disponible.
Cuándo NO aplicar	Equipos pequeños; requisitos cambiantes.	Sistemas de seguridad crítica; pliego SERCOP.

⚠ Advertencia / Error frecuente

Falacia frecuente. "Somos ágiles, no necesitamos documentar requisitos." Los métodos ágiles *no* eliminan la necesidad de especificar requisitos con criterios de aceptación verificables: la eliminan del documento monolítico y la distribuyen en el backlog. Un backlog sin criterios de aceptación es igualmente inútil que un ERS sin criterios de aceptación [**Cohn2004**, **Schwaber2020**, **Pohl2024**].

Ejemplo en Ecuador

¿Qué enfoque debe usar el PFC de la UTEQ?
La respuesta depende de tres factores:

1. **Disponibilidad del cliente**: si el cliente (dueño de la camaronera, mayordomo de la hacienda) puede reunirse semanalmente, el enfoque ágil con Scrum + BDD es más eficiente.
2. **Requisito del pliego SERCOP**: si el PFC es parte de una contratación pública, el pliego exige un ERS formal (IR tradicional).
3. **Criticidad**: para el sistema de aulas IoT de la UTEQ con control de acceso, se recomienda un enfoque híbrido: ERS formal para los RF de seguridad (IEEE 29148); backlog ágil para los RF de funcionalidad de usuario.

El **enfoque híbrido** (ERS mínimo + backlog) es la recomendación para la mayoría de PFC de la UTEQ [**Pohl2024**, **IREB_CPRE_2024**].

1.5 Mapa síntesis de la Semana 14

Tabla 1.5. *Mapa de cierre de la Semana 14.*

Subtema	Concepto clave	Artefacto del PFC
14.1 Backlog ágil	Épica→Historia→Tarea; INVEST; refinement.	Backlog priorizado del PFC en Jira.
14.2 BDD / Gherkin	Feature, Scenario, Given/When/Then; step defs.	3+ feature files con criterios ejecutables.
14.3 Scrum + IR	PO como analista; IR en cada evento Scrum.	Sprint Backlog del sprint 1 del PFC.
14.4 XP vs. Ágil	Customer on site; comparación 8 dimensiones.	Decisión documentada de enfoque para el PFC.

1.6 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 14 (en clase, 2 horas).

Taller BDD: de historia de usuario a feature file ejecutable.

1. Tomar 3 historias del PFC del equipo y escribir sus feature files completos en Gherkin (mínimo 2 escenarios por historia, incluyendo flujos alternativos y un Scenario Outline).
2. Implementar los step definitions vacíos en Python (Behave) o Java (Cucumber): solo la firma de cada step, sin lógica de negocio.
3. Ejecutar los tests en modo "pendiente" y verificar que el framework los reconoce correctamente.
4. Completar la tabla de comparación IR tradicional vs. ágil para el PFC propio: ¿qué enfoque usarán y por qué?

Práctica / Actividad

Tarea autónoma n.º 14 (4 horas, equipo del PFC).

1. **Backlog completo en Jira**: todas las épicas del PFC descompuestas en historias INVEST con criterios GWT y puntos de historia estimados.
2. **Feature files**: al menos 5 feature files cubrien los RF más críticos del PFC, con step definitions vacíos en el lenguaje del proyecto.
3. **Sprint Backlog del Sprint 1**: definir cuáles historias entran al primer sprint de implementación, con la Definition of Done del equipo.
4. **Decisión de enfoque**: documento de 1 página justificando la elección entre IR tradicional, IR ágil o híbrido para el PFC, con referencia a los tres factores de decisión.

1.7 Resumen ejecutivo de la semana

- El **Product Backlog** es el artefacto central de IR en Scrum; su calidad depende del PO y de los criterios de aceptación [**Schwaber2020**, **Cohn2004**].
- **BDD** convierte los criterios de aceptación en especificaciones ejecutables que sirven simultáneamente como pruebas de regresión automáticas [**Adzic2011**, **North2006**].
- En Scrum, **cada evento tiene actividades de IR**: el Refinement es la sesión de análisis de requisitos, la Review es la validación [**Schwaber2020**, **Pohl2024**].
- La IR ágil **no elimina** los requisitos: los distribuye en el tiempo y los mantiene en el backlog en lugar de en un documento monolítico [**Cohn2004**, **Pohl2024**].
- El **enfoque híbrido** (ERS mínimo + backlog) es la recomendación práctica para la mayoría de PFC de la UTEQ [**IREB_CPRE_2024**].